

TECHNICAL ARTICLE

How to Read a PostgreSQL Execution Plan

A Complete Guide with Real Examples, Diagnostics, Troubleshooting, and Production Fixes

Validated against PostgreSQL 18 official documentation

Prepared as an operations-focused performance reference | September 2026

Scope Covers EXPLAIN options, plan-tree reading, scans, joins, sorts, aggregation, parallelism, buffers, I/O, WAL, row-estimation errors, real labs, production diagnostics, and safe remediation. Example plans and timings vary by data, statistics, settings, cache, and hardware.

Table of Contents

1. Executive Summary	3
2. What an Execution Plan Represents	3
3. Safe EXPLAIN Command Patterns	4
4. How to Read a Plan Tree	5
5. Costs, Rows, Width, Time, and Loops	5
6. Scan Nodes	6
7. Join Nodes	7
8. Sort, Aggregate, Hash, and Utility Nodes	8
9. Parallel Query Nodes	10
10. Buffers, I/O, WAL, Memory, and Serialization	10
11. Real Lab and Plan Examples	11
12. Troubleshooting Workflow	13
13. Common Performance Findings and Fixes	14
14. Production Monitoring	14
15. Common Mistakes and Corrections	15
16. Production Checklist	16
17. Official-Documentation Validation	17
18. References	17

1. Executive Summary

A PostgreSQL execution plan is a tree of operations selected by the query planner. Each node consumes rows from its child nodes, performs work, and returns rows to its parent. Reading a plan correctly means following the tree from the most-indented nodes upward, while comparing what the planner estimated with what execution actually did.

Core method Start with total execution time, then find row-estimation errors, high-loop nodes, large buffer or temporary I/O, expensive sorts or hashes, rows removed by filters, heap fetches, and parallel-worker differences. Do not judge a plan only by its top node or by the presence of an index.

EXPLAIN ANALYZE executes the statement. Use it only when execution is safe. For data-changing statements, wrap the test in BEGIN and ROLLBACK, while remembering that locks, triggers, sequences, external side effects, and runtime load still require careful change control.

2. What an Execution Plan Represents

Part	Meaning	Operational question
Plan node	One physical operation such as a scan, join, sort, or aggregate.	What work does this node perform?
Child node	Input provider shown beneath and indented.	What rows feed the parent?
Estimated values	Planner predictions before execution.	Were statistics and assumptions accurate?
Actual values	Observed timing, rows, loops, buffers, and other metrics.	Where did runtime work accumulate?
Planning summary	Planning time, settings, and optionally planner memory.	Was planning itself expensive or affected by settings?
Execution summary	Executor time; optional serialization time is separate.	Does server execution explain user-visible latency?

Important Planner costs are unitless estimates used to compare candidate plans. They are not milliseconds and should not be compared directly with actual elapsed time.

3. Safe EXPLAIN Command Patterns

Command	Executes query?	Use
EXPLAIN	No	Inspect estimated plan safely.
EXPLAIN (ANALYZE)	Yes	Compare estimates with actual rows and timing.
BUFFERS	With ANALYZE, yes	Show shared/local/temp block activity and I/O details where enabled.
WAL	With ANALYZE, yes	Show WAL records, full-page images, bytes, and buffer-full events.
SETTINGS	No additional execution	Show nondefault settings relevant to planning.
MEMORY	No additional execution	Show planner memory, not executor work_mem consumption.
SERIALIZE	With ANALYZE, yes	Measure output conversion, still excluding network transfer.
FORMAT JSON/YAML/XML	Depends on ANALYZE	Produce machine-readable plan output.

```
-- Estimated plan only
EXPLAIN (VERBOSE, COSTS, SETTINGS)
SELECT * FROM orders WHERE customer_id = 1001;

-- Safe SELECT runtime plan
EXPLAIN (ANALYZE, BUFFERS, WAL, SETTINGS, SUMMARY)
SELECT * FROM orders WHERE customer_id = 1001;

-- Lower timing overhead when row counts/buffers are enough
EXPLAIN (ANALYZE, BUFFERS, TIMING OFF, SUMMARY)
SELECT * FROM orders WHERE customer_id = 1001;

-- Data-changing statement: executes, then rolls back table changes
BEGIN;
EXPLAIN (ANALYZE, BUFFERS, WAL, SETTINGS)
UPDATE orders SET status='processed' WHERE order_id=5001;
ROLLBACK;
```

Safety ROLLBACK does not make every test consequence-free. Sequence values are not rolled back, volatile/external functions may have side effects, locks are taken, triggers execute, WAL may be generated, and production load is still affected.

4. How to Read a Plan Tree

Text plans are executed conceptually from the deepest indentation toward the left. Sibling nodes provide separate inputs to a parent. The arrow marks child nodes; indentation shows ownership, not chronological timestamps.

```
Hash Join
  Hash Cond: (o.customer_id = c.customer_id)
    -> Seq Scan on orders o
        Filter: (order_date >= CURRENT_DATE - 30)
    -> Hash
        -> Seq Scan on customers c
            Filter: active
```

1. Read the deepest children first: orders and customers are scanned.
2. Check conditions at the node where they are applied: scan filters differ from join conditions.
3. The Hash node builds an in-memory or batched hash table from customers.
4. The Hash Join probes that structure using rows from orders.
5. Finish at the top node to understand final output and total server execution.

Plan ownership Node time is generally inclusive of child-node work. Do not add every node time together; doing so double-counts nested execution.

5. Costs, Rows, Width, Time, and Loops

Field	How to read it	Main warning
cost=a..b	Startup cost a; total cost b for expected complete output.	Unitless planner comparison, not elapsed time.
rows	Estimated rows produced per node execution.	Compare with actual rows, considering loops.
width	Estimated average bytes per output row.	Wide rows increase memory, I/O, CPU, and transfer work.
actual time=a..b	Average startup and completion time per loop in text plans.	Multiply carefully when loops > 1; timing has overhead.
actual rows	Average rows returned per loop.	Total output is approximately rows x loops.
loops	Number of node executions.	A cheap inner node may dominate when repeated many times.

Field	How to read it	Main warning
Planning Time	Parse/rewrite excluded; planner optimization time shown.	Can rise with many joins, partitions, or planning complexity.
Execution Time	Executor startup/shutdown, triggers as documented; output/network caveats apply.	Client-observed time can be higher.

```
-- Quick estimate-error calculation from a captured plan:
-- estimation factor = greatest(actual_rows / estimated_rows,
--                             estimated_rows / actual_rows)
-- Handle zero separately; review loops before calculating totals.
```

Priority signal Large row-estimation errors near the bottom of the tree often cause bad join order, join type, memory sizing, and scan choices above them.

6. Scan Nodes

Node	Typical use	What to inspect
Seq Scan	Large fraction of table, small table, or no useful index.	Rows removed by filter, buffers, I/O, table size, selectivity.
Index Scan	Selective lookup or index-provided ordering.	Index Cond vs Filter, heap fetch pattern, loops, correlation.
Index Only Scan	Needed columns in index and visibility map helps.	Heap Fetches; included columns; all-visible coverage.
Bitmap Index Scan	Collect tuple locations, possibly combine indexes.	Estimated/actual matches and bitmap input.
Bitmap Heap Scan	Visit matching heap pages in physical order.	Heap Blocks exact/lossy, Recheck Cond, rows removed by recheck.
Tid / Tid Range Scan	Direct tuple identifier access.	Specialized usage and actual tuple count.
Subquery / Function / Values Scan	Rows from derived source or function.	Function cost, estimates, repeated evaluation, materialization.
Foreign Scan	Remote or foreign data wrapper access.	Remote SQL, rows transferred, pushdown, network/remote latency.

6.1 Index Cond, Recheck Cond, and Filter

- Index Cond navigates or restricts index entries.
- Recheck Cond verifies bitmap matches at the heap, especially for lossy bitmap pages or access-method semantics.
- Filter is applied after rows are produced by the access path; many rows removed can reveal an incomplete index or predicate mismatch.

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT order_id, customer_id, order_date
FROM orders
WHERE customer_id = 1001
AND order_date >= CURRENT_DATE - INTERVAL '30 days';
```

6.2 Index-only scan nuance

An Index Only Scan does not promise zero heap reads. PostgreSQL may visit the heap when the visibility map cannot confirm that a page is all-visible. Write-heavy tables may therefore show many Heap Fetches even when the index covers every selected column.

7. Join Nodes

Join	How it works	Best fit / risk
Nested Loop	For each outer row, execute or probe the inner input.	Excellent with small outer input and fast inner lookup; dangerous with underestimated outer rows and high loops.
Hash Join	Build hash table from one input, probe with the other.	Good equality join; may batch/spill if memory is insufficient or estimates are wrong.
Merge Join	Consume inputs in join-key order.	Good for large ordered inputs; sorts can be expensive or spill.
Semi Join	Return outer row when a match exists.	EXISTS/IN patterns; inner search can stop at first match.
Anti Join	Return outer row when no match exists.	NOT EXISTS patterns; validate NULL semantics when rewriting NOT IN.
Outer Join	Preserve unmatched rows from required side.	Filters placed after join can accidentally change semantics or work.

7.1 Nested Loop diagnosis

```
Nested Loop (actual rows=50000 loops=1)
  -> Seq Scan on customers (actual rows=10000 loops=1)
  -> Index Scan on orders_customer_id_idx
      (actual rows=5 loops=10000)
```

The inner scan looks fast per execution, but it runs 10,000 times. Review total rows, buffer activity, cache behavior, and whether the outer estimate caused the planner to choose this join.

7.2 Hash join diagnosis

```
Hash Join
  Hash Cond: (o.customer_id = c.customer_id)
  -> Seq Scan on orders o
  -> Hash
      Buckets: 131072 Batches: 8 Memory Usage: 4096kB
  -> Seq Scan on customers c
```

Multiple batches indicate the hash table did not fit in its available memory and used batch processing, often involving temporary I/O. Do not raise `work_mem` globally without calculating concurrency and the number of memory-consuming nodes per query.

7.3 Merge join diagnosis

Merge Join requires ordered inputs. The order can come from indexes or Sort nodes. Check whether sorts spill, whether existing indexes can provide useful order, and whether the chosen join reduces total work.

8. Sort, Aggregate, Hash, and Utility Nodes

Node	Purpose	Evidence of trouble
Sort	Order rows for ORDER BY, merge join, grouping, or windowing.	External merge / Disk, high rows x width, repeated loops.
Incremental Sort	Exploit a prefix already ordered, sort within groups.	Unexpected group sizes or insufficient presorted benefit.
Aggregate	Compute aggregate; strategy may be plain, sorted, or hashed.	Large groups, spills, inaccurate group estimates.
Group / GroupAggregate	Process grouped ordered input.	Expensive upstream sort or repeated work.
Hash / HashAggregate	Build hash structure for joins or grouping.	Batches, disk usage, memory pressure.

Node	Purpose	Evidence of trouble
Unique	Remove duplicates, often from sorted input.	Large sort or unnecessary DISTINCT.
Limit	Stop after required rows.	Child may still do large startup work; compare startup cost.
Materialize / Memoize	Cache intermediate rows or parameterized results.	Memory use, evictions/overflows, low cache hit ratio.
WindowAgg	Evaluate window functions.	Large partitions, sorting, wide rows, temp I/O.
Append / Merge Append	Combine partitions or relations.	Too many scanned partitions; check pruning and per-child work.
CTE Scan	Read materialized or inlined CTE result as planned.	Repeated scans or optimization boundary assumptions.
LockRows / ModifyTable	Row locking or DML operation.	Lock waits, trigger time, WAL, rows affected.

8.1 Sort methods

```
Sort Method: quicksort  Memory: 2048kB
Sort Method: external merge  Disk: 512000kB
Sort Method: top-N heapsort  Memory: 128kB
```

An external sort used temporary storage. A top-N heapsort is optimized for LIMIT. Sort time can also reflect slow child input, so inspect both the Sort node and its child.

8.2 work_mem warning

Capacity rule work_mem is not a per-query limit. A query can have several sort/hash nodes, parallel workers may each allocate memory, and many sessions run concurrently. Test session-level changes before considering broader configuration.

9. Parallel Query Nodes

Node/field	Meaning
Gather	Collect unordered rows from workers; leader may also perform parallel work.

Node/field	Meaning
Gather Merge	Collect already ordered streams while preserving order.
Workers Planned	Workers chosen during planning.
Workers Launched	Workers actually obtained at execution.
Parallel Seq/Index/Bitmap Heap Scan	Cooperating processes scan the relation.
Partial Aggregate	Worker-local aggregation before results reach leader.
Finalize Aggregate	Leader combines partial results.

```
Gather (actual rows=... loops=1)
  Workers Planned: 4
  Workers Launched: 2
  -> Parallel Seq Scan on fact_sales
      Filter: (sale_date >= CURRENT_DATE - 30)
```

When fewer workers launch than planned, the selected plan may run slower than expected. Review `max_worker_processes`, `max_parallel_workers`, `max_parallel_workers_per_gather`, concurrent worker demand, and whether the leader is overloaded collecting a large output.

10. Buffers, I/O, WAL, Memory, and Serialization

Metric	Meaning	Do not assume
shared hit	Block found in PostgreSQL shared buffers.	Zero physical work overall; OS and prior reads still matter.
shared read	Block brought into shared buffers during execution.	Each block equals one slow device read; prefetch/cache layers exist.
dirtied / written	Buffers changed or written by this backend.	All write cost appears at the same node or completes synchronously.
local	Temporary-table buffer activity.	Same as shared buffers.
temp read/written	Temporary files used by sorts, hashes, materialization, etc.	<code>work_mem</code> is always the only cause.
I/O timings	Measured read/write timing when tracking is enabled.	All elapsed time is storage I/O.

Metric	Meaning	Do not assume
WAL records/fpi/bytes	Write-ahead log generated by executed statement.	WAL bytes equal table bytes changed.
Planning Memory	Planner structures and allocation overhead.	Executor memory or total backend RSS.
Serialization	Output conversion to selected representation.	Client network transfer is included.

```
SHOW track_io_timing;
EXPLAIN (ANALYZE, BUFFERS, WAL, SETTINGS, MEMORY,
         SERIALIZE TEXT, SUMMARY)
SELECT * FROM large_result_query;
```

Latency boundary EXPLAIN ANALYZE does not send result rows to the client. Without SERIALIZE it also omits output-conversion work. Application latency can therefore be higher because of serialization, network transfer, driver behavior, and client processing.

11. Real Lab and Plan Examples

11.1 Lab schema and data

```
DROP TABLE IF EXISTS orders, customers;
CREATE TABLE customers(
  customer_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  country char(2) NOT NULL,
  active boolean NOT NULL DEFAULT true
);
CREATE TABLE orders(
  order_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id bigint NOT NULL REFERENCES customers,
  order_date date NOT NULL,
  status text NOT NULL,
  amount numeric(12,2) NOT NULL
);

INSERT INTO customers(country,active)
SELECT (ARRAY['US','UK','CA','AU','IE'])[1+floor(random()*5)::int],
       random() < 0.9
FROM generate_series(1,50000);

INSERT INTO orders(customer_id,order_date,status,amount)
SELECT 1+floor(random()*50000)::bigint,
       CURRENT_DATE-floor(random()*730)::int,
       (ARRAY['new','paid','shipped','cancelled'])[1+floor(random()*4)::int],
       round((10+random()*990)::numeric,2)
FROM generate_series(1,100000);

ANALYZE customers; ANALYZE orders;
```

11.2 Example A - selective lookup before and after index

```
EXPLAIN (ANALYZE, BUFFERS, SETTINGS)
SELECT * FROM orders
WHERE customer_id=25000
  AND order_date >= CURRENT_DATE-90;

CREATE INDEX CONCURRENTLY orders_customer_date_idx
ON orders(customer_id, order_date);

ANALYZE orders;
EXPLAIN (ANALYZE, BUFFERS, SETTINGS)
SELECT * FROM orders
WHERE customer_id=25000
  AND order_date >= CURRENT_DATE-90;
```

Compare scan type, estimated/actual rows, buffers, and total time. The index may be highly effective because both conditions reduce the search range. Timings should be repeated under comparable cache conditions.

11.3 Example B - low-selectivity predicate

```
CREATE INDEX CONCURRENTLY orders_status_idx ON orders(status);
ANALYZE orders;
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders WHERE status='paid';
```

If approximately one quarter of a wide table matches, a sequential or bitmap plan can be cheaper than a simple Index Scan. Index existence does not require index use.

11.4 Example C - join and aggregation

```
EXPLAIN (ANALYZE, BUFFERS, SETTINGS)
SELECT c.country, count(*) AS orders, sum(o.amount) AS revenue
FROM customers c
JOIN orders o USING(customer_id)
WHERE o.order_date >= CURRENT_DATE-90
  AND c.active
GROUP BY c.country
ORDER BY revenue DESC;
```

6. Read scans first and compare their row estimates.
7. Check join type, build/probe side, hash batches, and buffers.
8. Inspect aggregate strategy and number of groups.
9. Inspect Sort method and whether temporary disk was used.
10. Check parallel workers planned versus launched.
11. Review the top node and total Execution Time last.

11.5 Example D - write plan with rollback

```
BEGIN;
EXPLAIN (ANALYZE, BUFFERS, WAL, SETTINGS)
UPDATE orders
SET status='shipped'
```

```
WHERE status='paid' AND order_date < CURRENT_DATE-30;
ROLLBACK;
```

Review rows selected, heap/index maintenance, buffers, WAL, trigger timing, and lock impact. Run only in a controlled environment or approved production window.

12. Troubleshooting Workflow

12. Capture the exact SQL, bind values, plan mode, PostgreSQL version, settings, and runtime context.
13. Check Planning Time, Execution Time, and SERIALIZE/client gap where relevant.
14. Read deepest nodes upward and mark every scan, join, sort, aggregate, and loop.
15. Compare estimated versus actual rows at each important node.
16. Multiply actual rows and work by loops where required.
17. Inspect Index Cond, Recheck Cond, Filter, Rows Removed, Heap Fetches, and partition pruning.
18. Inspect shared/local/temp buffers, I/O timing, WAL, sort method, hash batches, and memory evidence.
19. Check parallel workers planned versus launched and leader bottlenecks.
20. Correlate with statistics freshness, pg_stats distributions, table/index size, locks, CPU, I/O, and concurrency.
21. Test the smallest evidence-based fix with representative data and stable cache conditions.
22. Measure write/storage/maintenance impact and keep a rollback plan.
23. Monitor plan and latency after production rollout.

13. Common Performance Findings and Fixes

Finding	Likely causes	Evidence-based response
Estimated rows far below actual	Stale stats, skew, correlated predicates, parameter sensitivity.	ANALYZE; higher targeted stats; extended stats; query/index redesign; test prepared-plan behavior.
High-loop inner scan	Nested loop chosen for underestimated outer rows.	Fix estimates/index; evaluate join alternatives; reduce outer rows.
Many rows removed by filter	Predicate not applied by access path.	Composite/partial/expression index where valid; make predicate indexable; reduce rows earlier.
External sort / temp I/O	Rows x width exceed memory; estimate error; repeated sort.	Reduce rows/width; use useful order; session-level work_mem test; avoid global guess.

Finding	Likely causes	Evidence-based response
Hash batches > 1	Hash table exceeded memory or estimates.	Correct estimates; reduce build side; test memory; review join/index design.
Index Only Scan with heap fetches	Visibility map not sufficiently all-visible.	Maintain VACUUM; consider workload churn; do not force index-only expectation.
Workers launched < planned	Worker pool exhausted or configured low.	Review worker settings and concurrency; tune carefully.
Fast EXPLAIN, slow application	Network, serialization, result size, locks, client, connection pool.	Use SERIALIZE; measure application phases; return fewer rows.
High planning time	Many joins/partitions, invalidation, complex planning.	Check prepared statements, partition design, GEQO threshold, and query complexity.
High WAL for DML	Many rows/indexes, full-page images, wide changes.	Batch carefully; remove unused indexes; plan checkpoints and replication capacity.

14. Production Monitoring

14.1 Find expensive normalized statements

```
SELECT queryid, calls,
       round(total_exec_time::numeric,2) AS total_ms,
       round(mean_exec_time::numeric,2) AS mean_ms,
       rows, shared_blks_hit, shared_blks_read,
       temp_blks_read, temp_blks_written,
       left(query,200) AS query
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 20;
```

14.2 auto_explain guarded example

```
-- Requires module availability and suitable privileges.
LOAD 'auto_explain';
SET auto_explain.log_min_duration = '2s';
SET auto_explain.log_analyze = on;
SET auto_explain.log_buffers = on;
SET auto_explain.log_timing = off;
SET auto_explain.log_nested_statements = on;
SET auto_explain.sample_rate = 0.05;
```

Overhead warning auto_explain.log_analyze can add per-node instrumentation overhead to all statements in scope, including statements that do not cross the logging threshold. Use narrow scope, sampling, timing off where appropriate, and production change control.

14.3 Check statistics and relation size

```
SELECT schemaname, tablename, attname, n_distinct,
       most_common_vals, most_common_freqs, correlation
FROM pg_stats
WHERE tablename IN ('orders', 'customers');

SELECT relname, reltuples::bigint, relpages,
       pg_size_pretty(pg_total_relation_size(oid)) AS total_size
FROM pg_class
WHERE oid IN ('orders'::regclass, 'customers'::regclass);
```

15. Common Mistakes and Corrections

Mistake	Correct view
Read the plan top to bottom.	Understand output from deepest children upward, then finish at the root.
Highest cost node is the slowest node.	Costs are estimates and node timing is inclusive; inspect actual work and children.
Add all actual node times.	This double-counts nested work. Node times are generally inclusive.
actual rows is always total rows.	Text output commonly reports average per loop; consider loops.
Index Scan is always better than Seq Scan.	Planner chooses based on selectivity, pages, ordering, correlation, cache, and costs.
shared hit means storage was never involved.	It means found in shared buffers during this execution; system history and OS cache are separate.
Heap Blocks exact equals physical disk reads.	Bitmap page accounting is not disk-read accounting; use BUFFERS read/hit and I/O timing.
EXPLAIN ANALYZE is safe because output is discarded.	The statement executes and side effects occur; only SELECT output rows are discarded.
ROLLBACK removes every effect.	Sequences, external effects, locks during execution, WAL, and workload impact need consideration.
work_mem is a query-wide cap.	It applies per eligible operation, with parallel and concurrent multiplication.
Workers Planned equals Workers Launched.	Runtime availability can reduce launched workers.

Mistake	Correct view
Execution Time equals application latency.	Network and client work are omitted; serialization is optional.

16. Production Checklist

- ☐ Exact SQL and representative parameters are captured.
- ☐ Plain EXPLAIN was reviewed before executing ANALYZE on risky statements.
- ☐ Estimated and actual rows were compared with loops considered.
- ☐ Plan was read from deepest child nodes toward the root.
- ☐ Filters, rows removed, rechecks, heap fetches, and partition pruning were checked.
- ☐ Shared/local/temp buffers and I/O timing were reviewed.
- ☐ Sort methods, hash batches, disk use, and memory risks were reviewed.
- ☐ Parallel workers planned versus launched were checked.
- ☐ Statistics, data skew, correlation, and extended-statistics needs were assessed.
- ☐ Server execution was separated from serialization, network, and client time.
- ☐ Fix was tested with representative scale and repeatable conditions.
- ☐ Index write cost, WAL, storage, vacuum, and rollback were assessed.
- ☐ Post-change monitoring and plan regression detection are in place.

17. Official-Documentation Validation

Concept	Validated result
Plan is a tree read through indentation	Correct.
Costs are milliseconds	Incorrect; costs are arbitrary planner units.
EXPLAIN ANALYZE executes the statement	Correct; side effects occur unless separately controlled.
BEGIN/ROLLBACK is recommended for DML plan testing	Correct, with sequence/external/load caveats.
Actual node times can be summed	Incorrect; nested timing is inclusive.

Concept	Validated result
Actual rows and time require loop awareness	Correct; text plans report per-loop averages for node details.
BUFFERS explains cache and block activity	Correct; it does not alone prove device latency.
WAL reports records, FPIs, and bytes	Correct when ANALYZE and WAL are enabled.
MEMORY reports query executor memory	Incorrect in PostgreSQL 18 EXPLAIN; it reports planner memory.
SERIALIZE includes network transfer	Incorrect; it measures output conversion, not network.
Gather always launches planned workers	Incorrect; runtime worker availability can be lower.
auto_explain analyze is free	Incorrect; instrumentation can have significant overhead.

18. References

- PostgreSQL 18 - Using EXPLAIN: <https://www.postgresql.org/docs/18/using-explain.html>
 - PostgreSQL 18 - EXPLAIN Command: <https://www.postgresql.org/docs/18/sql-explain.html>
 - PostgreSQL 18 - Planner Statistics: <https://www.postgresql.org/docs/18/planner-stats.html>
 - PostgreSQL 18 - Row Estimation Examples: <https://www.postgresql.org/docs/18/row-estimation-examples.html>
 - PostgreSQL 18 - Planner Cost Constants: <https://www.postgresql.org/docs/18/runtime-config-query.html#RUNTIME-CONFIG-QUERY-CONSTANTS>
 - PostgreSQL 18 - How Parallel Query Works: <https://www.postgresql.org/docs/18/how-parallel-query-works.html>
 - PostgreSQL 18 - Parallel Plans: <https://www.postgresql.org/docs/18/parallel-plans.html>
 - PostgreSQL 18 - pg_stat_statements: <https://www.postgresql.org/docs/18/pgstatstatements.html>
 - PostgreSQL 18 - auto_explain: <https://www.postgresql.org/docs/18/auto-explain.html>
 - PostgreSQL 18 - Indexes: <https://www.postgresql.org/docs/18/indexes.html>
 - Source article - Kiran Sabne: <https://kiransabne.dev/how-to-read-a-postgresql-execution-plan-the-complete-guide-with-real-examples>
- Validation basis: PostgreSQL 18 official documentation was used as the primary authority. The linked article supplied the topic and learning direction; where third-party wording could not be independently retrieved, this guide does not attribute unverified claims to it. Confirm the documentation for the deployed major version before production changes.